MATHEMATICS / INDEPENDENT STUDY

# MM8036

# Type Theory

Written lectures · eight teaching units

Original teaching material aligned with Stockholm University course descriptions.
Independently authored; not an official university publication.

Edition 2 · 25 October 2026

# Contents

1 Unit 1 · Binding and substitution ..... 3
1.1 Lecture 1 · Lambda terms and free variables ..... 3
1.2 Lecture 2 · Capture-avoiding substitution ..... 4
2 Unit 2 · Reduction and confluence ..... 6
2.1 Lecture 3 · Beta reduction and evaluation strategies ..... 6
2.2 Lecture 4 · Parallel reduction and Church–Rosser ..... 7
3 Unit 3 · Simple types and judgments ..... 10
3.1 Lecture 5 · Typing as an inductive judgment ..... 10
3.2 Lecture 6 · Substitution, preservation, and progress ..... 11
4 Unit 4 · Normalization and constructive logic ..... 14
4.1 Lecture 7 · Strong normalization by reducibility ..... 14
4.2 Lecture 8 · Curry–Howard and the BHK interpretation ..... 15
5 Unit 5 · Dependent types ..... 18
5.1 Lecture 9 · Contexts, judgments, and dependent products ..... 18
5.2 Lecture 10 · Dependent sums and constructive witnesses ..... 19
6 Unit 6 · Induction, identity, and universes ..... 22
6.1 Lecture 11 · Natural numbers and dependent induction ..... 22
6.2 Lecture 12 · Identity elimination and universe levels ..... 23
7 Unit 7 · Substitutions and semantics ..... 26
7.1 Lecture 13 · Explicit substitutions as context morphisms ..... 26
7.2 Lecture 14 · Meaning explanations and a family-of-sets model ..... 27
8 Unit 8 · Categorical models and proof assistants ..... 29
8.1 Lecture 15 · Products, exponentials, and comprehension ..... 29
8.2 Lecture 16 · Formal proofs as checked programs ..... 30

# 1 Unit 1 · Binding and substitution

## 1.1 Lecture 1 · Lambda terms and free variables

**Learning goals and prerequisites**
Parse lambda terms, distinguish binding from application, and compute free variables. Prerequisites: inductive definitions and elementary logic.

**Recall and approach.** Recall inductively generated trees and that one variable name may occur both freely and bound in different subterms.

The untyped lambda calculus isolates two operations: constructing a function and applying it. Terms are generated by variables, application $tu$, and abstraction $\lambda x.t$. Application associates to the left, so $tuv$ means $(tu)v$. An abstraction's body extends as far right as parentheses permit. Thus $\lambda x.xy$ is a function with body $xy$.

**Definition 1 · Free variables**
Define $\mathrm{FV}(x) = \{x\}$, $\mathrm{FV}(tu) = \mathrm{FV}(t) \cup \mathrm{FV}(u)$, and $\mathrm{FV}(\lambda x.t) = \mathrm{FV}(t) \setminus \{x\}$. A term is closed if it has no free variables. An occurrence of $x$ is bound by the nearest enclosing abstraction naming $x$, if one exists.

**Example 1 · Shadowing**
In $\lambda x.(x(\lambda x.x))y$, the first occurrence of $x$ is bound by the outer abstraction and the occurrence inside $\lambda x.x$ by the inner one. The variable $y$ is free. Renaming the inner binder first makes the structure clearer: $\lambda x.(x(\lambda z.z))y$.

An inductive definition of terms gives a structural induction principle: prove a property for every variable, prove it is inherited by an application from its two subterms, and prove it is inherited by an abstraction from its body. This is different from induction on a reduction sequence, which will concern computations rather than syntax trees.

**Definition 2 · Alpha equivalence**
Renaming a bound variable consistently to a fresh name does not change the binding structure. Alpha equivalence is the equivalence generated by such renamings and their closure under term formation. For example $\lambda x.x$ and $\lambda z.z$ are alpha equivalent, but $\lambda x.y$ and $\lambda y.y$ are not.

**Proof strategy.** Use structural induction and keep the binding relation invariant under fresh renaming.

**Theorem 1 · Free variables respect alpha equivalence**
Alpha-equivalent terms have the same free variables.

**Proof.** It suffices to check one capture-free renaming and its placement inside a larger term. In a renamed abstraction, occurrences bound by that abstraction change name, and the new name is chosen fresh for the body; none becomes free and no previously free occurrence becomes bound. Thus removing the bound variable from the body's free-variable set gives the same result. The application and abstraction context cases follow from the recursive equations for FV. Reflexivity, symmetry, and transitivity preserve equality of free-variable sets. □

Named variables are readable but require freshness management. An alternative is de Bruijn indexing: a bound variable records how many binders outward one must go to find its binder. This removes alpha-

renaming choices but requires careful index shifts under binders. We retain names in the mathematical development and make freshness conditions explicit.

**Example 2 · Reading a term before manipulating it**
In $(\lambda x.xy)(\lambda y.y)$, the left abstraction binds only its occurrence of $x$. Its $y$ is free. The right abstraction binds a different occurrence of the name $y$. The entire term therefore has free-variable set $\{y\}$. Renaming the right binder to $z$ makes this separation visible without changing the term's alpha class.

To calculate, mark each abstraction's body first, then connect each bound occurrence to its nearest matching binder. Only after those connections are clear should a substitution be attempted. A string search for a letter cannot distinguish these scopes.

**Checkpoint 1.1.** Compute $\mathrm{FV}(\lambda x.(xz)(\lambda z.yz))$.

**Takeaways.** Read the syntax tree before performing a substitution or reduction. Identical variable names can denote different bound occurrences.
Continue with study guide U1 and exercise sheet U1.

## 1.2 Lecture 2 · Capture-avoiding substitution

**Learning goals and prerequisites**
Define substitution recursively, prove its free-variable bound, and apply the substitution-composition law with its freshness condition.

**Recall and approach.** Recall the free-variable set and the unchanged-term lemma when the replaced variable is absent.

**Definition 3 · Substitution**
$t[x := u]$ replaces the free occurrences of $x$ in $t$ by $u$. For variables, $x[x := u] = u$ and $y[x := u] = y$ when $y \neq x$. Substitution distributes over applications. Under $\lambda x$, it stops. Under $\lambda y$ with $y \neq x$, first ensure $y \notin \mathrm{FV}(u)$ by alpha-renaming if necessary, then substitute in the body.

**Example 3 · Why renaming is necessary**
In $(\lambda y.x)[x := y]$, the inserted $y$ must remain free. Rename the binder to fresh $z$ first, obtaining $\lambda z.y$. The expression $\lambda y.y$ would incorrectly capture the inserted variable and turn a constant function into the identity.

**Proof strategy.** Induct on syntax, renaming each binder fresh before passing substitution underneath it.

**Theorem 2 · Free-variable bound**
$\mathrm{FV}(t[x := u]) \subseteq (\mathrm{FV}(t) \setminus \{x\}) \cup \mathrm{FV}(u)$. If $x \notin \mathrm{FV}(t)$, substitution leaves $t$ unchanged up to alpha equivalence.

**Proof.** Induct on $t$. For a variable, use the two defining cases. For application, use the induction bounds on its two parts and take their union. For an abstraction binding $x$, substitution stops and no free $x$ occurs in the abstraction. For another binder $y$, choose it fresh for $u$; apply the induction hypothesis to its body, then remove $y$ from the free-variable set. Freshness ensures this removal does not delete a free variable newly inserted from $u$. The unchanged-term assertion follows from the same induction. □

**Theorem 3 · Substitution composition**
If $x \neq y$ and $x \notin \mathrm{FV}(v)$, then, up to alpha equivalence,

$$(t[x := u])[y := v] = (t[y := v])[x := u[y := v]]. \tag{1}$$

All bound names are chosen fresh for the terms being substituted.

**Proof.** Induct on $t$. If $t = x$, both sides are $u[y := v]$. If $t = y$, the left side is $v$, while the right is $v[x := u[y := v]] = v$ by the freshness assumption. Any other variable is unchanged. The application case follows componentwise. For an abstraction, first rename its binder to avoid $x, y$ and all free variables in $u, v$; substitution then passes under it on both sides, and the induction hypothesis proves equality of the bodies. □

The freshness assumption is essential. With $t = y$ and $v = x$, the left side gives $x$ while the right side may substitute inside that newly inserted $x$. Composition laws are about controlled substitution, not arbitrary textual replacement.

**Example 4 · Composition records which occurrence was inserted**
Take distinct variables $x, y, z, w$, with $t = xy$, $u = yz$, and $v = w$. Substituting $u$ for $x$ and then $v$ for $y$ gives $(wz)w$. Reversing the two steps naively would give $(yz)w$. The correct composition law also substitutes $v$ into the replacement $u$, giving $wz$ before it is inserted.

The equation is therefore more than a convenient rearrangement of notation: it tracks whether a later substitution should act inside an earlier replacement. Its side condition prevents an occurrence inserted by the second substitution from being changed incorrectly on the other side.

**Checkpoint 1.2.** Compute $(\lambda y.xy)[x := y]$.

**Takeaways.** Substitution is the operation behind both computation and dependent typing. Establish capture avoidance now; later metatheorems rely on it at every binder.
Continue with study guide U1 and exercise sheet U1.

# 2 Unit 2 · Reduction and confluence

**Bridge into this unit.** A reduction step changes one redex, while a conversion can travel forward and backward through many steps. To compare arbitrary reduction paths, first build a relation that reduces compatible pieces together. Confluence concerns joining paths; termination concerns whether a path can continue forever.

## 2.1 Lecture 3 · Beta reduction and evaluation strategies

**Learning goals and prerequisites**
Distinguish reduction from evaluation, identify normal forms, and separate weak from strong normalization.

**Recall and approach.** Recall full beta reduction may act anywhere, while a weak strategy does not reduce under abstractions.

**Definition 4 · Beta reduction**
A beta redex is $(\lambda x.t)u$, whose contraction is $t[x := u]$. One-step beta reduction allows one such contraction at any position, including inside an abstraction. Write $t \underset{\beta}{\to} u$ and write $\overset{*}{\underset{\beta}{\to}}$ for its reflexive transitive closure. A beta normal form contains no beta redex.

**Example 5 · Two routes through one computation**
The term $(\lambda x.x)((\lambda y.y)z)$ can first contract the outer redex to $(\lambda y.y)z$, then to $z$. Alternatively reduce the argument to $z$, then contract $(\lambda x.x)z$. Both routes reach $z$, but they pass through different intermediate terms.

**Definition 5 · Normalization distinctions**
A term is weakly normalizing if some finite reduction sequence reaches a normal form. It is strongly normalizing if no infinite reduction sequence starts from it. A reduction strategy chooses which available redex to contract; it is additional structure beyond the unrestricted reduction relation.

Let $\Delta = \lambda x.xx$ and $\Omega = \Delta\Delta$. Contracting its root redex returns $\Omega$, giving an infinite reduction. Yet $(\lambda x.y)\Omega$ has normal form $y$ by contracting the outer redex immediately. Reducing inside the argument forever gives an infinite path. Hence weak and strong normalization are genuinely different properties of a term.

Call-by-name evaluates the function position and substitutes an argument without first evaluating it. Call-by-value first evaluates the function and argument to values before contracting their application. Both are usually weak evaluation strategies, meaning they do not reduce under an abstraction. Their "finished values" need not be beta normal forms under unrestricted reduction.

**Proof strategy.** Check a root contraction with the substitution bound and lift the argument through surrounding syntax.

**Theorem 4 · Reduction does not introduce new free variables**
If $t \underset{\beta}{\to} u$, then $\mathrm{FV}(u) \subseteq \mathrm{FV}(t)$.

**Proof.** For a root redex, the substitution bound gives $\mathrm{FV}(s[x := v]) \subseteq (\mathrm{FV}(s) \setminus \{x\}) \cup \mathrm{FV}(v)$, exactly the free variables of $(\lambda x.s)v$. For a step within an application or abstraction, use the induction hypothesis for the changed subterm and the recursive equations for FV. □

**Watch for this.** An unrestricted reduction relation need not be deterministic. A deterministic evaluation strategy does not establish that every possible reduction order has the same behavior. Confluence, proved next, is the property that relates different choices.

**Example 6 · A value can still contain a redex**
The term $\lambda z.(\lambda x.x)z$ is a value under weak call-by-value because it is an abstraction. Under full beta reduction it steps inside its body to $\lambda z.z$, so it is not a beta normal form. Replacing the body by $\Omega$ gives a weak value with infinitely many full reductions.

When reporting a computation, state both the allowed positions and the rule for choosing among them. A strategy can stop because it refuses to enter a body, even though the unrestricted relation has available steps there. This distinction matters when comparing implementation behavior with a theorem about all reductions.

**Checkpoint 2.1.** Is $\lambda z.\Omega$ a value under a weak call-by-value strategy? Is it a beta normal form?

**Takeaways.** Whenever saying "reduces," specify the relation or strategy. Values, normal forms, and strongly normalizing terms are different notions.
Continue with study guide U2 and exercise sheet U2.

## 2.2 Lecture 4 · Parallel reduction and Church–Rosser

**Learning goals and prerequisites**
Prove confluence using parallel reduction and derive uniqueness of normal forms without claiming termination.

**Recall and approach.** Recall the substitution-composition lemma and the reflexive transitive closure of a relation.

**Definition 6 · Parallel reduction**
Write $t \Rightarrow u$ for the relation generated by variable reflexivity, congruence under abstraction and application, and the rule: if $s \Rightarrow s'$ and $v \Rightarrow v'$, then $(\lambda x.s)v \Rightarrow s'[x := v']$. It can contract several existing redexes in one step.

The relation is reflexive by structural induction. Every one-step beta reduction is a parallel step. Every parallel step is a finite sequence of beta steps: first reduce the component terms by induction, then contract the root redex if that parallel rule was used. Thus the two relations have the same reflexive transitive closure.

**Proof strategy.** Use parallel reduction to synchronize duplicated residual redexes, then transfer its diamond property to beta sequences.

**Theorem 5 · Parallel substitution lemma**
If $s \Rightarrow s'$ and $u \Rightarrow u'$, then $s[x := u] \Rightarrow s'[x := u']$, with capture avoidance.

**Proof.** Induct on the derivation of $s \Rightarrow s'$. Variable and congruence rules follow directly from substitution's definition. In the redex rule, rename the binder fresh for the substituted terms, apply the induction hypotheses to its body and argument, then use the parallel redex rule. The substitution-composition identity from Lecture 2 identifies the resulting contractum with the required substituted term. □

Define the complete development $t^*$ recursively: variables remain themselves; develop an abstraction's body; for an application that is already a root redex $(\lambda x.s)v$, use $s^*[x := v^*]$; for any other application $sv$, use $s^*v^*$.

**Theorem 6 · Complete-development lemma**
If $t \Rightarrow u$, then $u \Rightarrow t^*$. Consequently parallel reduction has the diamond property.

**Proof.** Induct on the syntax of $t$, considering the parallel rule used. Variable and abstraction cases are immediate. If $t$ is not a root redex, its parallel step uses application congruence; the induction hypotheses develop both resulting components to their starred versions, and application congruence gives $t^*$. If $t = (\lambda x.s)v$ and the step uses congruence, the result is $(\lambda x.s')v'$; the induction hypotheses on $s, v$ followed by the parallel redex rule reach $s^*[x := v^*]$. If it uses root contraction, the result is $s'[x := v']$, and the parallel substitution lemma with those same induction hypotheses reaches the identical term. Thus any two parallel successors of $t$ both parallel-reduce to $t^*$. □

**Theorem 7 · Confluence of beta reduction**
If $t \xrightarrow[\beta]{*} u$ and $t \xrightarrow[\beta]{*} v$, there is $w$ with $u \xrightarrow[\beta]{*} w$ and $v \xrightarrow[\beta]{*} w$. Hence beta normal forms are unique up to alpha equivalence.

**Proof.** The diamond property extends to finite sequences by filling a rectangular grid of one-step diamonds, formally an induction on the two sequence lengths. Parallel and beta reduction have the same transitive closure, so the result transfers to beta reduction. If $u, v$ are normal forms, neither can take a genuine further beta step, so a common reduct forces them to be alpha equivalent. □

**Example 7 · Why the one-step diamond is the wrong target**
In $(\lambda x.xx)((\lambda y.y)z)$, contracting the argument first permits one more step to $zz$. Contracting the outside first duplicates the argument redex and generally needs two ordinary steps to the same term. Thus a proof demanding that the two beta successors join by exactly one ordinary step would fail. Parallel reduction can contract both residual copies together, which is why its diamond property is better suited to the proof.

For the conversion form of Church–Rosser, induct along a finite zigzag of forward and reversed beta steps. Maintain a common reduct with the initial endpoint; a forward step uses confluence to join the old common reduct with the new step, and a reversed step extends the new endpoint's path. This yields a common reduct for convertible terms.

**Checkpoint 2.2.** Why does confluence not prove that $\Omega$ has a normal form?

### 2.2.1 A slower route through the central ideas

**Example 8 · A duplicated argument and two first steps**
Let $a = (\lambda y.y)z$. Starting from $(\lambda x.xx)a$, contracting the outside redex gives $aa$, then reducing its two copies gives $zz$. Reducing the original argument first gives $(\lambda x.xx)z$, then contracting gives the same $zz$. The first route duplicates a redex, so a one-step diamond is not enough: one branch may need more than one ordinary step to catch up. Parallel reduction handles the two copied occurrences together.

**Expanded critical case for confluence.** The parallel substitution lemma says that $s \Rightarrow s'$ and $u \Rightarrow u'$ imply $s[x := u] \Rightarrow s'[x := u']$. Its proof is structural induction on s, with fresh renaming in the abstraction case. This lemma is necessary because contracting a root redex substitutes the whole argument, possibly at several occurrences.

Define the complete development of a term recursively, developing the body and argument of every originally present redex before substituting. The triangle lemma states that any parallel reduct of t has a parallel reduction to this complete development. For a root redex, a branch that contracts it has form $s'[x := u']$; induction on the constituents and the substitution lemma join it to the developed contractum. A branch that leaves the outer application in place can contract that outer redex on its next parallel step. Nonredex applications and abstractions use the corresponding induction hypotheses componentwise. These cases establish the triangle and hence the diamond for parallel reduction.

Every ordinary step is a parallel step, and every parallel step can be implemented by finitely many ordinary steps. Iterating the parallel diamond therefore yields confluence for ordinary multistep beta reduction. Church–Rosser then turns a finite zigzag of forward and backward steps into a common reduct.

**Example 9 · One normal form does not force every strategy to finish**
Put $I = \lambda z.z$ and $\Omega = (\lambda u.uu)(\lambda u.uu)$. The term $(\lambda x.I)\Omega$ contracts at the root to I, a normal form. Yet full beta reduction may keep reducing the Omega subterm to itself forever. This term is weakly normalizing but not strongly normalizing. Confluence says that any normal form it reaches is the same as I up to alpha equivalence; it does not remove the infinite sequence.

**A failed shortcut.** Checking that two chosen reductions meet is an example, not a confluence proof. The structural lemmas must cover arbitrary terms, substitution, and every possible reduction rule. Conversely, confluence alone is not a proof of normalization.

**Takeaways.** Confluence is a compatibility theorem about branching computation. Normalization is a separate termination theorem.
Continue with study guide U2 and exercise sheet U2.

# 3 Unit 3 · Simple types and judgments

**Bridge into this unit.** Typing is an inductively generated judgment, so proofs about every typed term proceed by its last typing rule. Substitution must preserve the information in the context. Preservation uses that lemma at the beta-redex case, while progress is a separate analysis of closed typed syntax.

## 3.1 Lecture 5 · Typing as an inductive judgment

**Learning goals and prerequisites**
Build typing derivations, distinguish terms from judgments, and prove weakening for the simply typed calculus.

**Recall and approach.** Recall implication introduction discharges an assumption and application requires a function and a matching argument type.

The simply typed lambda calculus assigns a type to each term according to rules. Our normalization core has types generated by a base type $o$ and arrows $A \to B$, with arrows associating to the right. We may include one constant $c : o$; it has no reduction rule. There is no unrestricted fixed-point operator.

**Definition 7 · Context and typing**
A context $\Gamma$ is a finite list of distinct variables with simple types. The judgment $\Gamma \vdash t : A$ is generated by the variable rule, the constant rule, abstraction, and application. It asserts that $t$ has type $A$ under the assumptions in $\Gamma$.

$$\frac{\Gamma\ ,\ x : A \vdash t : B}{\Gamma \vdash (\lambda x.t) : A \to B}$$

Arrow introduction

$$\frac{\Gamma \vdash f : A \to B \quad \Gamma \vdash u : A}{\Gamma \vdash fu : B}$$

Arrow elimination

In the variable rule, if $x : A$ occurs in $\Gamma$, then $\Gamma \vdash x : A$. The constant has type $o$ in every valid context. A derivation is a finite tree of rule applications, and induction on that tree is the appropriate proof method for most typing lemmas.

**Example 10 · Composition**
Under $f : B \to C, g : A \to B, x : A$, application first gives $gx : B$ and then $f(gx) : C$. Abstract successively over $x, g, f$ to obtain the closed term $\lambda f.\lambda g.\lambda x.f(gx)$ of type $(B \to C) \to (A \to B) \to A \to C$.

**Proof strategy.** Induct on typing derivations; each constructor is handled by rebuilding the same rule from its transformed premises.

**Theorem 8 · Weakening**
If $\Gamma \vdash t : A$ and $\Gamma'$ extends $\Gamma$ with fresh assumptions without changing existing assignments, then $\Gamma' \vdash t : A$.

**Proof.** Induct on the typing derivation. A variable still has its original assignment, and the constant rule remains available. The application case follows from the two induction hypotheses. For abstraction, rename the bound variable fresh for the enlarged context, apply the induction hypothesis to the body derivation, and reapply abstraction. □

**Theorem 9 · Typable terms use only declared variables**
If $\Gamma \vdash t : A$, then $\mathrm{FV}(t)$ is contained in the variables declared by $\Gamma$.

**Proof.** Induct on the derivation. Variables and constants satisfy the claim. Application takes a union of free-variable sets. Abstraction removes its bound variable, which is exactly the additional assumption discharged by that rule. □

**Watch for this.** In the Curry-style syntax used here, an unannotated term may admit many simple types; for instance the identity has $A \to A$ for any $A$. A fixed derivation specifies a type, but a general uniqueness-of-type claim would require an annotated syntax or additional qualifications.

**Example 11 · An explicit derivation tree for repeated application**
Under $f : A \to A, x : A$, variable rules give both declarations. Application derives $fx : A$, then a second application derives $f(fx) : A$. Abstracting over $x$ yields $f : A \to A \vdash \lambda x.f(fx) : A \to A$; abstracting over $f$ gives $(A \to A) \to A \to A$.

The repeated occurrence of $f$ must use the same declaration each time. This is where typing becomes a constraint on syntax rather than a label attached independently to each occurrence. If the first application returned a different type, it could not automatically be fed back into $f$.

**Checkpoint 3.1.** Why can $\lambda x.xx$ not be typed with finite simple types?

**Takeaways.** Types are syntax; typing is a judgment justified by rules; a derivation is evidence for that judgment. Keep these levels separate.
Continue with study guide U3 and exercise sheet U3.

## 3.2 Lecture 6 · Substitution, preservation, and progress

**Learning goals and prerequisites**
Prove the typed substitution lemma, derive subject reduction, and establish progress for a specified call-by-value strategy.

**Recall and approach.** Recall weakening permits unused assumptions but does not permit changing an existing variable type.

**Proof strategy.** Prove substitution first; use inversion for preservation and canonical forms for closed progress.

**Theorem 10 · Typed substitution**
If $\Gamma, x : A \vdash t : B$ and $\Gamma \vdash u : A$, then $\Gamma \vdash t[x := u] : B$, with capture avoidance. More generally, extra simple-type assumptions may be retained by weakening.

**Proof.** Induct on the first derivation. The variable $x$ is replaced by the given derivation of $u$; every other variable is unchanged. The constant case is immediate. For application, apply the induction hypotheses to function and argument and reapply the rule. For abstraction $\lambda y.s$, rename $y$ fresh for $u, \Gamma, x$. Weaken the derivation of $u$ to include $y$ and apply the induction hypothesis to the body, then abstract over $y$. Simple types contain no term variables, so the type $B$ itself does not change under this substitution. □

**Theorem 11 · Preservation / subject reduction**
If $\Gamma \vdash t : A$ and $t \underset{\beta}{\to} t'$, then $\Gamma \vdash t' : A$.

**Proof.** For a root redex $(\lambda x.s)u$, inversion of the application and abstraction rules gives a body derivation with assumption $x : B$ and an argument of type $B$. Typed substitution derives the contractum at the same result type. For a reduction inside application or abstraction, apply induction to the changed subderivation and reconstruct the enclosing typing rule. □

For progress, specify weak call-by-value evaluation. Values are abstractions and the base constant. An application first steps its function part, then its argument once the function is a value, and contracts a beta redex only when the argument is a value. There are no steps under abstractions in this strategy.

**Theorem 12 · Canonical forms and progress**
A closed value of arrow type is an abstraction. Every closed well-typed term is either a value or can take a call-by-value step.

**Proof.** The canonical-forms assertion follows by inspecting the two value forms: the constant has only base type, so a value of arrow type must be an abstraction. For progress, induct on a closed term's typing derivation. The variable case is impossible in an empty context. A constant or abstraction is a value. For an application, use induction on its function: if it steps, so does the application. Otherwise it is a value; use induction on the argument. If the argument steps, so does the application. If both are values, canonical forms makes the function an abstraction, so the beta-value rule applies. □

Progress and preservation together rule out reaching a closed stuck nonvalue by this evaluation strategy. They do not by themselves prove termination. An unrestricted recursive construct can preserve types and progress while diverging; our normalization theorem will use the more restrictive syntax of this core.

**Example 12 · Where a progress proof needs its hypotheses**
In context $f : o \to o, x : o$, the term $fx : o$ is well typed. Neither variable can step, and the function position is not an abstraction, so the application is stuck under the specified strategy. This is not a counterexample to progress because the theorem requires the empty context.

Conversely, a closed typable function value cannot be the base constant: its type is an arrow while the constant's declared type is $o$. This tiny canonical-forms lemma is the bridge from "the function finished evaluating" to "a beta rule is available." Merely knowing that both subterms are values is insufficient without that bridge.

**Checkpoint 3.2.** Why is closedness necessary for this progress statement?

### 3.2.1 A slower route through the central ideas

**Example 13 · A complete small typing derivation**
In context $f : o \to o, x : o$, the variable rule gives $f : o \to o$ and $x : o$. Application gives $fx : o$; abstraction first over x and then over f gives the closed term $\lambda f.\lambda x.fx$ of type $(o \to o) \to o \to o$.

$$\frac{f : o \to o, x : o \vdash f : o \to o \quad f : o \to o, x : o \vdash x : o}{f : o \to o, x : o \vdash fx : o}$$

Application

$$\frac{f : o \to o, x : o \vdash fx : o}{f : o \to o \vdash \lambda x.fx : o \to o}$$

Abstraction

**Expanded substitution proof.** Suppose $\Gamma, x : A \vdash t : B$ and $\Gamma \vdash u : A$. Induct on the derivation for t. If t is x, the desired judgment is exactly the supplied typing of u. For another variable or the constant, substitution leaves it unchanged. If the last rule is application, the two induction hypotheses preserve the function's arrow type and the argument's domain type, so application reconstructs the result. If it is abstraction $\lambda y.s$, rename y fresh for u and x. Weaken the typing of u to the context extended by y, apply the induction hypothesis to s, and abstract y again. This is where freshness and context bookkeeping prevent capture.

For a typed redex $(\lambda x.s)u$, inversion of application and abstraction typing supplies exactly the two premises of substitution. Its contractum $s[x := u]$ therefore has the same result type. Reductions in subterms use induction and reconstruct the outer typing rule, proving preservation for full beta reduction.

**Example 14 · Following the type through two reductions**
The term $(\lambda f.\lambda x.fx)(\lambda z.z)$ has type $o \to o$: the function expects an argument of type $o \to o$, and the identity has that type. Contracting the outside redex gives $\lambda x.(\lambda z.z)x$, still of type $o \to o$. Reducing under its binder gives $\lambda x.x$ of the same type. The substituted function's type is what makes the inner application legal at each stage; the visible syntax change does not alter the final judgment.

**A failed shortcut.** Self-application $xx$ would require the same simple type A of x to satisfy $A = A \to B$. A finite simple type cannot contain itself as a proper subtree. This obstruction comes from the typing rules, not from merely observing that one particular self-application diverges.

**Takeaways.** Substitution is the engine of preservation. Canonical forms are the engine of progress. Termination requires a separate argument.
Continue with study guide U3 and exercise sheet U3.

# 4 Unit 4 · Normalization and constructive logic

**Bridge into this unit.** Induction on typing supplies termination of subterms, but termination is not stable under arbitrary application. Reducibility strengthens the invariant to control applications at each type. The proof's three layers are candidate closure, controlled beta expansion, and the fundamental substitution lemma.

## 4.1 Lecture 7 · Strong normalization by reducibility

**Learning goals and prerequisites**
Understand why induction on typing needs a strengthened invariant and prove strong normalization for the arrow-only simply typed core.

**Recall and approach.** Recall absence of infinite reduction, induction on types, and the syntax-directed typing rules.

The desired statement is that no infinite full beta-reduction sequence begins at a typable term. A direct application-case induction fails: two terminating terms can form a nonterminating application, as $\Delta\Delta$ shows. We therefore strengthen termination to a property indexed by the term's type.

**Definition 8 · Reducible terms**
Let SN be the strongly normalizing raw terms. Define $\mathcal{R}_o = \mathrm{SN}$ and

$$\mathcal{R}_{A\to B} = \{t \in \mathrm{SN} : \text{for every } u \in \mathcal{R}_A, tu \in \mathcal{R}_B\}. \tag{2}$$

Reducibility is defined for raw terms; the fundamental lemma will connect it to typing. A neutral term here means a term that is not an abstraction, so applications, including beta redexes, are neutral.

**Proof strategy.** Strengthen termination to a type-indexed application property, and separate candidate closure, expansion, and the fundamental lemma.

**Theorem 13 · Candidate properties**
For every type $A$: (1) reducibility implies strong normalization; (2) a reduct of a reducible term is reducible; (3) a neutral term all of whose immediate reducts are reducible is reducible.

**Proof.** Induct on $A$. At the base type these are the corresponding elementary properties of absence of an infinite reduction path. At an arrow type, (1) is part of the definition. For (2), a reduct $t'$ remains strongly normalizing, and $tu \underset{\beta}{\to} t'u$; closure at the result type puts $t'u$ in $\mathcal{R}_B$ for every reducible $u$. For (3), the hypothesis makes every reduct of $t$ strongly normalizing, hence $t$ strongly normalizing. Fix reducible $u$. Prove $tu \in \mathcal{R}_B$ by induction on the maximum reduction length of $u$, which exists because reduction is finitely branching and $u$ is strongly normalizing. Since $t$ is not an abstraction, a one-step reduct of $tu$ reduces either $t$ or $u$. Reducing $t$ gives $t'u$ reducible by the hypothesis at arrow type. Reducing $u$ gives $tu'$ reducible by the smaller-length induction. Neutral closure at $B$ now gives the claim. □

In particular every variable is reducible at every type: it is neutral and has no reducts. The same applies to the inert base constant at its base type.

**Theorem 14 · Controlled beta expansion**
If $s, u$ are strongly normalizing and $s[x := u] \in \mathcal{R}_B$, then $(\lambda x.s)u \in \mathcal{R}_B$.

**Proof.** Induct on the sum of the maximal reduction lengths of $s, u$. The root contraction is reducible by assumption. A step in $s$ gives $s'$ with $s'[x := u]$ a reduct of the substituted body, hence reducible by

closure; the induction hypothesis handles the corresponding redex. A step $u \to u'$ makes $s[x := u']$ a finite reduct of $s[x := u]$, reducing each substituted occurrence (or taking zero steps when none occur), so it is reducible; use induction again. These exhaust the immediate reducts. The redex is neutral under our definition, so neutral closure at $B$ proves the result. □

**Theorem 15 · Fundamental lemma and normalization**
If $\Gamma \vdash t : A$ and a simultaneous capture-avoiding substitution sends every $x : B$ in $\Gamma$ to a term in $\mathcal{R}_B$, then the substituted term lies in $\mathcal{R}_A$. Consequently every term typable in the arrow-only core is strongly normalizing.

**Proof.** Induct on the typing derivation. Variables and the constant follow from the assumptions; application follows from the arrow definition. For an abstraction $\lambda x.s$, rename $x$ fresh for the substitution. Extending the environment by a fresh reducible variable shows the substituted body is strongly normalizing, by the induction hypothesis and alpha-renaming. For arbitrary reducible argument $u$, extend the environment by $x := u$; induction makes the substituted contractum reducible. Controlled beta expansion therefore makes the application reducible. The abstraction itself is strongly normalizing because its body is, so it satisfies the arrow clause. Finally choose the identity substitution: each variable is reducible, so $t$ is reducible and hence strongly normalizing. □

**Example 15 · Why the proof uses raw terms**
The reducibility sets are defined on raw terms so that substitution environments and neutral closure can be handled uniformly. A variable has no reducts, so it belongs to every candidate by neutral closure. This does not mean a variable has every type in a fixed context: candidate membership is a metatheoretic predicate, while typing is a separate inductive judgment. The fundamental lemma is precisely the theorem connecting the two.

To read the abstraction case, make two columns: the fresh-variable environment proves that the body terminates; an arbitrary reducible-argument environment proves the contractum reducible. Controlled beta expansion combines those facts. Omitting the first column leaves one hypothesis of the expansion lemma unproved.

**Checkpoint 4.1.** Why is the arrow reducibility condition stronger than simply asking the function term to terminate?

**Takeaways.** The strengthened invariant makes application work; the expansion lemma makes abstraction work. The theorem here concerns the specified simply typed core, not every later dependent type theory or every Agda feature.
Continue with study guide U4 and exercise sheet U4.

## 4.2 Lecture 8 · Curry–Howard and the BHK interpretation

**Learning goals and prerequisites**
Translate constructive propositions into types and proofs into terms, including products, sums, and the empty type.

**Recall and approach.** Recall natural deduction, assumption discharge, and the difference between constructing data and denying its impossibility.

Under Curry–Howard, a typing derivation can be read as a natural-deduction proof. An assumption $x : A$ is available evidence for proposition $A$; a term of type $A \to B$ transforms evidence for $A$ into evidence for $B$. Abstraction discharges an assumption, and application is implication elimination.

**Definition 9 · Constructive connectives**
Conjunction is represented by a product $A \times B$, with pair introduction and two projections. Disjunction is a sum $A + B$, with injections naming the chosen alternative and case analysis as elimination. Truth is the unit type $1$ with one constructor. Falsehood is the empty type $0$ with no constructors; its eliminator produces a term of any type from an assumed term of $0$. Negation abbreviates $A \to 0$.

The Brouwer–Heyting–Kolmogorov interpretation explains what constitutes evidence: a conjunction proof supplies both pieces; a disjunction proof supplies a chosen side and its evidence; an implication proof supplies a uniform transformation; an existential proof supplies a witness and evidence about it; a universal proof supplies evidence uniformly for each input. Dependent sums and products will make the last two precise.

**Example 16 · Two proof programs**
A proof of $A \to B \to A$ is $\lambda a.\lambda b.a$. A proof of $(A \times B) \to (B \times A)$ takes a pair and returns its two projections in reversed order. A proof of $(A + B) \to C$ requires a function $A \to C$ and a function $B \to C$, then uses case analysis on the tag.

**Proof strategy.** Read the introduction and elimination rules of each connective as constructors and consumers of evidence.

**Theorem 16 · Constructive distributivity**
There are maps both ways between $A \times (B + C)$ and $(A \times B) + (A \times C)$ whose composites compute to the identity on constructor inputs.

**Proof.** Given $(a, s)$, inspect $s$. If it is a left injection of $b$, return the left injection of $(a, b)$; if a right injection of $c$, return the right injection of $(a, c)$. Conversely inspect the outer sum and return the corresponding pair with a tagged second coordinate. On either constructor case, each composite returns the original constructor data by the product and sum computation rules. □

Computational content depends on the proof principles adopted. The general law $A + (A \to 0)$ is not an added axiom of our constructive core. A decision procedure for a particular proposition can nevertheless construct such a sum, selecting a branch with evidence. Classical principles, if later assumed, must be recorded explicitly rather than hidden inside a case distinction.

**Watch for this.** The normalization proof in the preceding lecture covered only base and arrow types. Products and sums can be treated by extending the reducibility argument with additional clauses; no normalization theorem for unrestricted recursion is being asserted here.

**Example 17 · One direction of a De Morgan principle**
A term of $(A + B) \to 0$ yields both $A \to 0$ and $B \to 0$: precompose it with the two injections. Conversely, a pair of refuters gives a refuter of $A + B$ by case analysis. Both constructions are constructive because a sum exposes its tag.

The corresponding informal proof should describe which evidence is consumed in each branch. A statement such as “one of the cases must hold” is not enough unless the input supplies a sum tag or a decision procedure produces one. This discipline makes the computational content of a proof explicit before any code is written.

**Checkpoint 4.2.** What extra information does a term of $A + B$ carry compared with the bare assertion that at least one side is true?

### 4.2.1 A slower route through the central ideas

**Expanded dependency map.** Define candidates by induction on types, with the arrow clause requiring both strong normalization and preservation of reducibility under application. Prove closure under reduction and neutral expansion by induction on type. In the arrow neutral case, fix a reducible argument and induct on its maximal reduction length. A reduct of the application reduces either the neutral function or its argument; the former uses the given arrow-candidate hypothesis on the function's reduct, while the latter uses the strictly smaller argument measure. Neutral closure at the result type then finishes the step. A strongly normalizing term has such a finite maximal length because this reduction system is finitely branching.

Controlled beta expansion next considers all immediate reducts of $(\lambda x.s)u$. They are the root contractum, a reduction in s, or a reduction in u. Induct on the sum of the two maximal reduction lengths. Candidate closure handles the corresponding substituted contracta, and neutral closure combines the cases. Here "neutral" means not an abstraction, so the redex itself qualifies; it does not mean beta-normal.

> **Example 18 · The identity satisfies every arrow self-candidate**
> Fix a type A. The raw body x is normal, hence strongly normalizing, so $\lambda x.x$ is strongly normalizing. For an arbitrary $u \in \mathcal{R}_A$, the contractum of $(\lambda x.x)u$ is u, already reducible. The body and argument both terminate, so controlled expansion makes the application reducible. Thus $\lambda x.x \in \mathcal{R}_{A\to A}$. This proof checks both arrow clauses rather than only the contraction's result.

**The abstraction case as two experiments.** In the fundamental lemma, first extend a reducible substitution by a fresh variable for the binder. The induction hypothesis makes the substituted body strongly normalizing. Next extend it by an arbitrary reducible argument. This makes the substituted contractum reducible. Controlled expansion combines the two experiments; neither can replace the other. Finally the identity substitution is reducible because variables lie in every candidate, connecting the substitution theorem back to all typed terms.

> **Example 19 · Why uncontrolled expansion is false**
> The raw term $(\lambda x.c)\Omega$ contracts to the reducible base constant c, but it is not strongly normalizing: reduction may remain inside Omega forever. Thus reducibility of a contractum alone does not justify reducibility of the redex. The controlled lemma's assumption that the argument terminates excludes this case. If a substitution erases its variable, that assumption is still required.

**A failed shortcut.** Candidate membership is a predicate on raw terms, not a typing judgment. A fresh variable belongs to every candidate without having every type in one fixed context. The fundamental lemma is the bridge from well-typed substitutions to these predicates. Its conclusion here is restricted to the specified arrow-only simply typed core.

**Takeaways.** Proof rules become term constructors, and proof simplification becomes computation. Optional Lab 1 implements these basic proof programs.
Continue with study guide U4 and exercise sheet U4.

# 5 Unit 5 · Dependent types

**Bridge into this unit.** A dependent function's result type changes with its input, while a dependent pair's second component is checked in the fiber selected by its first. Keep a term and its type together through every projection and substitution; ordinary tuple notation can hide the crucial dependency.

## 5.1 Lecture 9 · Contexts, judgments, and dependent products

**Learning goals and prerequisites**
Read dependent judgments, distinguish formation/introduction/elimination/computation rules, and type applications whose result depends on the argument.

**Recall and approach.** Recall contexts, alpha-renaming, and the simple substitution lemma; now types may also contain term variables.

In simple type theory, result types do not mention term variables. In dependent type theory they may: a vector's type can include its length, and a function can return a proof about the very input it receives. The type system must therefore track substitution in types as well as terms.

**Definition 10 · Judgment forms**
We use judgments that a context is valid, that $\Gamma \vdash A$ type, that $\Gamma \vdash t : A$, and judgmental equalities $\Gamma \vdash A \equiv B$ type and $\Gamma \vdash t \equiv u : A$. Equality here is a judgment of the theory, not yet a type inhabited by an equality proof.

A dependent context is an ordered list $x_1 : A_1, \ldots, x_n : A_n$, where each $A_i$ is a type in the preceding context. Reordering assumptions can be invalid. The abbreviation "$A$ type" will later mean that $A$ belongs to an appropriate universe. Our core uses intensional identity types, predicative universe levels, beta computation and congruence; it does not adopt equality reflection, uniqueness of identity proofs, function extensionality, or univalence as axioms. We do not require a function-eta judgmental rule in the paper core.

**Definition 11 · Dependent product**
If $\Gamma \vdash A$ type and $\Gamma, x : A \vdash B(x)$ type, then $\Pi_{x:A} B(x)$ is a type. Introduction takes a term $b(x) : B(x)$ in the extended context to $\lambda x.b(x) : \Pi_{x:A} B(x)$. Elimination takes $f : \Pi_{x:A} B(x)$ and $a : A$ to $f(a) : B(a)$. The computation rule is $(\lambda x.b(x))(a) \equiv b(a)$.

$$\frac{\Gamma \vdash f : \Pi_{x:A} B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)}$$

Dependent application

Ordinary function types are the case where $B$ does not depend on $x$. Under Curry–Howard, a dependent product expresses universal quantification: one term must provide evidence of $B(a)$ uniformly for every input $a$.

**Example 20 · An indexed identity function**
For a family $B : A \to U$, the expression $\lambda x.\lambda y.y$ has type $\Pi_{x:A}(B(x) \to B(x))$. After supplying $a : A$, it becomes the identity function on the particular fiber $B(a)$. The result type changes with the first argument.

**Proof strategy.** Track substitution in the result type as carefully as in the term; apply each rule only in a well-formed context.

**Definition 12 · Structural rules in this presentation**
Conversion permits a term of type $A$ to be used at a judgmentally equal type $B$. We take simultaneous substitution of well-typed terms as a structural rule: substituting $a : A$ for an assumption $x : A$ substitutes into both the term and all subsequent dependent types. Context formation and freshness must remain valid. This is part of our specified dependent theory; it is not a claim that substitution has been derived for every possible extension of the calculus.

**Theorem 17 · Substitution at a dependent application**
Let $f : \Pi_{x:A} B(x)$ and $a : A$ be over $\Gamma$, and let $\sigma : \Delta \to \Gamma$ be a well-typed substitution. Then $(fa)[\sigma]$ and $f[\sigma](a[\sigma])$ have the same dependent result type over $\Delta$.

**Proof.** Rename $x$ fresh for $\sigma$. Substituting into the Pi judgment gives $f[\sigma] : \Pi_{x:A[\sigma]} B[\sigma^+](x)$, where $\sigma^+$ extends $\sigma$ by retaining the fresh variable $x$. Applying to $a[\sigma]$ gives type $B[\sigma^+][x := a[\sigma]]$. Substituting into the original result judgment instead gives $(B[x := a])[\sigma]$. These type expressions agree by the capture-avoiding substitution-composition law, applied to their term and type syntax. Distribution of substitution through application identifies their term expressions. Conversion therefore places both at the same stated type. □

**Example 21 · Specializing a dependent identity**
Let $B : A \to U_0$ and define $f = \lambda x.\lambda y.y$. Its type is $\Pi_{x:A}(B(x) \to B(x))$. Applying it to $a : A$ gives a function on $B(a)$, not on an arbitrary fixed $B$. Applying that result to $b : B(a)$ computes to $b$.

Every step can be read from the result type: first substitute $a$ into the family, then use the resulting function domain. In a dependent derivation, exchanging declarations is legal only when it preserves these formation dependencies. The ordinary simple-type exchange rule cannot be copied without checking them.

**Checkpoint 5.1.** Why is the context $n : \mathrm{Nat}, v : \mathrm{Vec}(A, n)$ not freely reorderable?

**Takeaways.** Every elimination rule must show the result's exact dependent type. Suppressing the substitution there hides the main mathematical content.
Continue with study guide U5 and exercise sheet U5.

## 5.2 Lecture 10 · Dependent sums and constructive witnesses

**Learning goals and prerequisites**
Use dependent pairs to package witnesses with evidence, type their projections, and derive a constructive choice principle.

**Recall and approach.** Recall products and existential evidence; a Sigma pair stores its witness and the evidence in that witness's fiber.

**Definition 13 · Dependent sum**
From a type $A$ and family $B(x)$, form $\Sigma_{x:A} B(x)$. A pair $(a, b)$ has this type when $a : A$ and $b : B(a)$. Its projections satisfy $\mathrm{fst}(p) : A$ and $\mathrm{snd}(p) : B(\mathrm{fst}(p))$, with computation rules $\mathrm{fst}(a, b) \equiv a$ and $\mathrm{snd}(a, b) \equiv b$.

The dependent elimination rule allows a family $C(p)$ over such pairs. To define a term of $C(p)$ for arbitrary $p$, it suffices to give $d(a, b) : C((a, b))$ for arbitrary $a : A, b : B(a)$. The eliminator computes

on a constructed pair by returning $d(a, b)$. This is stronger than merely projecting components because the output type itself may depend on the whole pair.

**Example 22 · Existence includes a witness**
A term of $\Sigma_{n:\mathrm{Nat}} P(n)$ contains an actual natural number and evidence of the property at that number. The type $\Sigma_{n:\mathrm{Nat}} \mathrm{Vec}(A, n)$ packages a finite sequence with its length. It is different from fixing $n$ first and asking for a vector of that prescribed length.

**Proof strategy.** Construct the witness function by first projection and its evidence by second projection, then check the dependent conversion.

**Theorem 18 · Constructive dependent choice of witnesses**
There is a term of

$$\left(\Pi_{x:A} \Sigma_{y:B(x)} C(x, y)\right) \to \Sigma_{f:\Pi_{x:A} B(x)} \Pi_{x:A} C(x, f(x)). \tag{3}$$

It is constructed without adding a classical choice axiom.

**Proof.** Given $h$, define $f = \lambda x.\mathrm{fst}(h(x))$ and $p = \lambda x.\mathrm{snd}(h(x))$. The second projection has type $C(x, \mathrm{fst}(h(x)))$, which converts to $C(x, f(x))$ by the beta rule for $f$. Therefore $(f, p)$ inhabits the required dependent sum. Every witness is extracted from the supplied constructive evidence $h(x)$, rather than chosen from a bare existence assertion with erased witnesses. □

This illustrates why translating a theorem into types requires attention to the chosen interpretation of existence. The input is already a uniform function returning witness–proof pairs. A classical statement about arbitrary nonempty sets does not automatically provide that computational data.

**Theorem 19 · Dependent pair extensionality, propositional form**
Every $p : \Sigma_{x:A} B(x)$ can be proved equal, in the identity type introduced next, to the reconstructed pair $(\mathrm{fst}p, \mathrm{snd}p)$.

**Proof.** Use dependent elimination on $p$. In the constructor case $p = (a, b)$, both projections compute and the reconstructed pair becomes $(a, b)$. Reflexivity therefore supplies the equality proof. This proves a propositional eta law without adopting it as judgmental equality. □

**Example 23 · A common-witness obstruction**
From $\Sigma_{x:A}(B(x) \times C(x))$, one can extract two Sigma terms with the same witness. The reverse construction does not follow from two arbitrary Sigma terms: their witnesses may differ. In a set interpretation with $A = \{0, 1\}$, let $B$ be inhabited only at zero and $C$ only at one. Both separate existence types are inhabited, while the common-witness type is empty.

This is a useful diagnostic for an attempted proof: write the first components of the supplied pairs before trying to combine their second components. A mismatch of witnesses is a mathematical missing hypothesis, not a typographical inconvenience.

**Checkpoint 5.2.** Why is the second projection's type not simply a fixed type $B$?

### 5.2.1 A slower route through the central ideas

**Example 24 · Applying an indexed identity in two stages**
Assume $A : U_0$ and $B : A \to U_0$. In context $x : A, y : B(x)$, the variable rule gives $y : B(x)$. Abstracting y gives $\lambda y.y : B(x) \to B(x)$; abstracting x gives $f : \Pi_{x:A}(B(x) \to B(x))$. Given $a : A$ and $b : B(a)$, dependent application first gives $f(a) : B(a) \to B(a)$ and then $f(a)(b) : B(a)$. Beta computation returns b. The first input changes the domain expected by the second application.

$$\frac{\Gamma \vdash f : \Pi_{x:A}(B(x) \to B(x)) \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a) \to B(a)}$$

First dependent application

$$\frac{\Gamma \vdash f(a) : B(a) \to B(a) \quad \Gamma \vdash b : B(a)}{\Gamma \vdash f(a)(b) : B(a)}$$

Second application

**Expanded substitution bookkeeping.** Under a context substitution sigma, a dependent application has type $(B[x := a])[\sigma]$. Substituting into the function first gives a family with the binder x retained fresh, written $B[\sigma^+]$; applying to $a[\sigma]$ then gives $B[\sigma^+][x := a[\sigma]]$. Capture-avoiding substitution composition identifies these two type expressions. It is not enough merely to note that the term syntax distributes as $(fa)[\sigma] = f[\sigma](a[\sigma])$: the types must match as well.

**Example 25 · Unpacking and repacking a witness**
For $p : \Sigma_{x:A}B(x)$, put $a = \mathrm{fst}(p)$ and $b = \mathrm{snd}(p)$. Their types are $a : A$ and $b : B(a)$, so the pair $(a, b)$ is well typed in the original sum. A function $h : \Pi_{x:A}(B(x) \to C)$ can consume this package as $h(a)(b) : C$. On an input constructor $p = (u, v)$, the expression computes to $h(u)(v)$. This gives an explicit uncurrying operation without selecting a witness from an untagged existential claim.

**Expanded constructive-choice calculation.** Suppose $g : \Pi_{x:A}\Sigma_{y:B(x)}C(x, y)$. Define $f = \lambda x.\mathrm{fst}(g(x))$. Its type is $\Pi_{x:A}B(x)$. The term $\lambda x.\mathrm{snd}(g(x))$ has type $\Pi_{x:A}C(x, f(x))$, since f(x) computes to the first projection of g(x). Pairing these terms constructs the claimed choice data. No external choice principle is used: g already provides a witness and its evidence uniformly for each x.

**A failed shortcut.** The second projection of p is not a term of a fixed type independent of p. Nor do constructor computation rules alone imply a judgmental eta equality for an arbitrary function or record variable. State which equality is being claimed.

**Takeaways.** Dependent sums package data and its specification. Optional Lab 3 uses indexed data to make length information part of the type.
Continue with study guide U5 and exercise sheet U5.

# 6 Unit 6 · Induction, identity, and universes

**Bridge into this unit.** An eliminator is determined by a motive: the family of result types it must produce. Choose that family before the base and step cases. For identity elimination the endpoint must be allowed to vary; for natural numbers the computation equations determine which induction variable is useful.

## 6.1 Lecture 11 · Natural numbers and dependent induction

**Learning goals and prerequisites**
State induction with its motive, distinguish recursion from induction, and compute a proof by structural recursion.

**Recall and approach.** Recall recursion with a constant result type and the distinction between a variable and a visible constructor.

**Definition 14 · Natural-number rules**
The type Nat has constructors $\mathrm{zero} : \mathrm{Nat}$ and $\mathrm{suc} : \mathrm{Nat} \to \mathrm{Nat}$. For a family $P(n)$, a base case $z : P(\mathrm{zero})$, and a step $s : \Pi_{n:\mathrm{Nat}}(P(n) \to P(\mathrm{suc}(n)))$, induction gives $\mathrm{ind}(P, z, s, n) : P(n)$. It computes to $z$ at zero and to $s(n, \mathrm{ind}(P, z, s, n))$ at a successor.

The family $P$ is the motive: it specifies what the output type must be at each input. Recursion is the special case in which this family is constant. A proof by induction becomes a program whose recursive call is justified by a structurally smaller natural number.

**Example 26 · Addition and its two zero laws**
Define $\mathrm{add}(\mathrm{zero}, n) \equiv n$ and $\mathrm{add}(\mathrm{suc}(m), n) \equiv \mathrm{suc}(\mathrm{add}(m, n))$. The left-zero law computes judgmentally. The right-zero law $\mathrm{add}(n, \mathrm{zero}) = n$ requires induction for an arbitrary variable $n$, because reduction cannot inspect a variable to choose its constructor.

**Proof strategy.** Write the motive before the cases and make recursive calls only on the structurally smaller input.

**Theorem 20 · Right-zero proof**
For every $n$, there is an identity proof of $\mathrm{add}(n, \mathrm{zero}) = n$.

**Proof.** Use motive $P(n) = \mathrm{Id}_{\mathrm{Nat}(\mathrm{add}(n,\mathrm{zero}),n)}$. At zero, addition computes and reflexivity proves the claim. In the successor case, the induction hypothesis gives equality of $\mathrm{add}(n, \mathrm{zero})$ and $n$. Applying the function suc to that equality gives equality of their successors, which is the required goal after the addition computation rule. Function application to equality is derived by identity elimination in Lecture 12. □

**Definition 15 · Length-indexed vectors**
For a type $A$, the family $\mathrm{Vec}(A, n)$ has a constructor $\mathrm{nil} : \mathrm{Vec}(A, \mathrm{zero})$ and a constructor taking $a : A$ and $v : \mathrm{Vec}(A, n)$ to $\mathrm{cons}(a, v) : \mathrm{Vec}(A, \mathrm{suc}(n))$.

Appending a vector of length $m$ to one of length $n$ produces length $\mathrm{add}(m, n)$. Recurse on the first vector: the empty case returns the second vector, and the constructor case retains the head and recursively appends the tail. The resulting length matches the addition equations by judgmental conversion. No separate length-correctness theorem is required because the result type already states it.

Inductive definitions must respect positivity restrictions. A purported datatype constructor accepting a function from the datatype itself into the empty type can introduce negative self-reference and undermine normalization or consistency. The optional labs use ordinary strictly positive natural-number, sum, equality, and vector definitions; the type checker verifies their permitted forms and termination.

**Example 27 · Associativity follows the definition's recursion direction**
For fixed $b, c$, prove $(a + b) + c = a + (b + c)$ by induction on $a$, where addition recurses on its first argument. At zero both sides compute to $b + c$. At successor they compute to successors of the two sides of the induction hypothesis, so congruence of successor proves the step.

Inducting on a different variable is not wrong in principle, but the defining equations may stop reducing and require extra lemmas. Choosing an induction variable to match computation often produces the shortest proof. The optional lab uses exactly this alignment, with the result expressed as an identity type.

**Checkpoint 6.1.** Why can $\mathrm{add}(\mathrm{zero}, n)$ reduce when $\mathrm{add}(n, \mathrm{zero})$ is stuck for variable $n$?

**Takeaways.** Write the motive before the induction cases. Dependent induction ensures each recursive result has exactly the type needed at the smaller input.
Continue with study guide U6 and exercise sheet U6.

## 6.2 Lecture 12 · Identity elimination and universe levels

**Learning goals and prerequisites**
Distinguish judgmental equality from identity proofs, derive transport, and use predicative universes without type-in-type.

**Recall and approach.** Recall a type judgment is different from a term of an identity type; universes stratify type formation.

**Definition 16 · Identity type and path induction**
For $a, b : A$, form $\mathrm{Id}_{A(a,b)}$, with constructor $\mathrm{refl}_a : \mathrm{Id}_{A(a,a)}$. To define a term of a family $C(a, b, p)$ for $p : \mathrm{Id}_{A(a,b)}$, it suffices to give $d(a) : C(a, a, \mathrm{refl}_a)$. The eliminator $J$ returns that case when $p$ is reflexivity.

**Proof strategy.** Generalize the endpoints in the motive of J, reduce to reflexivity, and then supply the required canonical proof.

**Theorem 21 · Transport and equality operations**
For a family $B : A \to U$ and $p : \mathrm{Id}_{A(a,b)}$, there is a function $\mathrm{transport}_{B(p)} : B(a) \to B(b)$, computing to the identity for reflexivity. Identity types also admit symmetry, transitivity, and congruence under a function.

**Proof.** For transport, use path induction with motive $C(a, b, p) = B(a) \to B(b)$ and reflexive case the identity function. For symmetry, use motive $\mathrm{Id}_{A(b,a)}$ and reflexivity in the reflexive case. For transitivity, induct on one equality proof with the other endpoint and proof appropriately retained in the motive; the reflexive case returns the remaining proof. For congruence under $f : A \to D$, use motive $\mathrm{Id}_{D(f(a),f(b))}$, with reflexivity at the base case. Each operation's computation on reflexivity follows from the $J$ rule. □

A term $p : \mathrm{Id}_{A(a,b)}$ does not automatically make $a$ and $b$ judgmentally equal. The latter is a conversion judgment used by the type checker; the former is internal evidence which can be consumed by transport. We do not assume equality reflection. Nor does path induction assert that every equality proof with fixed endpoints is judgmentally reflexivity; that mistaken inference would confuse the general motive of $J$ with a stronger uniqueness principle.

**Definition 17 · Universe hierarchy**
We use levels $U_0 : U_1 : U_2 : \ldots$. If $A : U_i$ and $B(x) : U_j$, dependent products and sums belong to $U_{\max(i,j)}$. Identity types on $A : U_i$ lie in $U_i$, and natural numbers lie in $U_0$. We do not identify a universe with its own type and do not assume cumulativity in the paper presentation.

**Example 28 · Polymorphic identity**
The expression $\lambda A.\lambda x.x$ inhabits $\Pi_{A:U_0}(A \to A)$, a type in $U_1$. The outer quantification ranges over $U_0$, whose type is $U_1$; it is not a small endofunction on an unstratified "type of all types."

Agda uses `Set`, `Set₁`, and further levels for this hierarchy. Its conversion includes some additional eta rules compared with our minimal paper core; the labs use proofs valid in their common fragment. The labs run with `--safe --without-K`, so they do not bypass checking by postulates or rely on the K principle for identity proofs.

**Example 29 · Changing a fiber without changing data**
If $p : \mathrm{Id}_{\mathrm{Nat}(m,n)}$ and $v : \mathrm{Vec}(A, m)$, transport in the family $k \mapsto \mathrm{Vec}(A, k)$ yields a vector in $\mathrm{Vec}(A, n)$. No vector elements are inserted or removed. The equality evidence justifies treating the same dependent data in the endpoint fiber. When $p$ is reflexivity, the cast computes to $v$.

For an arbitrary proof variable $p$, that reflexive computation is not immediately available. A further proof about the cast should eliminate $p$ with a motive valid for varying endpoints, rather than pretending all paths are judgmentally reflexivity.

**Checkpoint 6.2.** When does transport compute immediately?

### 6.2.1 A slower route through the central ideas

**Example 30 · Computing a proof of the right-zero law**
Addition recurses on its first argument. Set $P(n) = \mathrm{Id}_{\mathrm{Nat}(\mathrm{add}(n,\mathrm{zero}),n)}$. At zero, both endpoints compute to zero, so reflexivity proves $P(\mathrm{zero})$. Given $h : P(n)$, congruence of successor gives an identity from $\mathrm{suc}(\mathrm{add}(n,\mathrm{zero}))$ to $\mathrm{suc}(n)$. The defining addition equation makes this precisely $P(\mathrm{suc}(n))$. At the concrete numeral two, both recursion steps unfold and the proof reduces through successor congruence; at a variable n, induction provides evidence where computation alone is stuck.

$$\frac{h : \mathrm{Id}_{\mathrm{Nat}(\mathrm{add}(n,\mathrm{zero}),n)}}{\mathrm{ap}_{\mathrm{suc}(h)} : \mathrm{Id}_{\mathrm{Nat}(\mathrm{suc}(\mathrm{add}(n,\mathrm{zero})),\mathrm{suc}(n))}}$$

Successor congruence

**Deriving congruence rather than assuming it.** For $f : A \to D$, use J with motive $C(a,b,p) = \mathrm{Id}_{D(f(a),f(b))}$. Its reflexive case is $\mathrm{refl}_{f(a)}$. Therefore $\mathrm{ap}_{f(p)}$ has the desired endpoints, and $\mathrm{ap}_{f(\mathrm{refl}_a)}$ computes to reflexivity. This supplies the equality operation used in the induction step above and makes its dependency explicit.

**Example 31 · A cast with a variable endpoint**
For $B : A \to U_0$, $a : A$, $b : A$, $p : \mathrm{Id}_{A(a,b)}$, and $v : B(a)$, the term v cannot simply be used at type $B(b)$. Take motive $C(a,b,p) = B(a) \to B(b)$ for J, with identity function as the reflexive case. This yields $\mathrm{transport}_{B(p)}(v) : B(b)$. When p is reflexivity, it computes to v. For a proof variable p it is a valid term in the new fiber but need not visibly reduce. In the vector family this changes the justified length index without adding or removing vector entries.

**A failed shortcut.** A proposed comparison of every loop p with reflexivity must first make sense for $p : \mathrm{Id}_{A(a,b)}$ at a general b. Reflexivity at a has the wrong identity type for that comparison. The missing formation judgment cannot be supplied by saying "now take b=a"; that restricts the elimination problem before applying J. This diagnoses the gap without asserting that J yields K or UIP.

**Universe check.** The identity family on $A : U_0$ remains in $U_0$. But the polymorphic type $\Pi_{A:U_0}(A \to A)$ lies in $U_1$, because its domain $U_0$ is itself a type in $U_1$. These level calculations are formation obligations, just as endpoint calculations are obligations for identity motives. No type-in-type or cumulativity rule is being added.

**Takeaways.** Equality proofs move terms between dependent fibers. Universe levels keep the formation rules stratified. Optional Lab 4 makes both transport and its computation visible.
Continue with study guide U6 and exercise sheet U6.

# 7 Unit 7 · Substitutions and semantics

## 7.1 Lecture 13 · Explicit substitutions as context morphisms

**Learning goals and prerequisites**
Represent simultaneous substitutions as typed objects, compose them in the correct direction, and explain substitution into dependent families.

**Recall and approach.** Recall simultaneous substitutions supply every declaration of the target context in order.

So far substitution has been a metalevel operation on syntax. Making substitutions explicit exposes the bookkeeping needed for implementation and categorical semantics. We use a typed substitution interface with equations, not a claim that every possible explicit-substitution rewrite system is confluent or terminating.

**Definition 18 · Context substitution**
For $\Gamma = (x_1 : A_1, \ldots, x_n : A_n)$, a substitution $\sigma : \Delta \to \Gamma$ is a tuple $(t_1, \ldots, t_n)$ with $\Delta \vdash t_1 : A_1$ and $\Delta \vdash t_i : A_i\left[\frac{t_1}{x_1}, \ldots, \frac{t_{i-1}}{x_{i-1}}\right]$. It interprets every variable of $\Gamma$ by a term in $\Delta$.

The arrow direction can initially feel reversed: a map from environments in $\Delta$ to environments in $\Gamma$ supplies values for all variables of $\Gamma$. Pulling a type or term back along it goes from syntax over $\Gamma$ to syntax over $\Delta$.

**Definition 19 · Identity and composition**
The identity substitution lists the context's own variables. Given $\rho : \Theta \to \Delta$ and $\sigma : \Delta \to \Gamma$, define $\sigma \circ \rho : \Theta \to \Gamma$ by substituting $\rho$ into each component of $\sigma$. Write $t[\sigma]$ for the resulting simultaneous substitution into a term, and similarly for types.

**Proof strategy.** Check equations componentwise and lift substitutions under binders by retaining the new bound variable.

**Definition 20 · Projection and extension of substitutions**
For the extended context $\Gamma \cdot A$, the projection substitution $p : \Gamma \cdot A \to \Gamma$ lists the old variables and forgets the last one. The generic term $q : A[p]$ is that last variable. Given $\sigma : \Delta \to \Gamma$ and $t : A[\sigma]$ over $\Delta$, append $t$ to the tuple $\sigma$ to obtain $\langle \sigma, t \rangle : \Delta \to \Gamma \cdot A$. Dropping its last component recovers $\sigma$, and looking up its last variable recovers $t$. These are the explicit tuple operations behind comprehension in Unit 8.

**Theorem 22 · Substitution laws**
Capture-avoiding simultaneous substitution satisfies $t[\mathrm{id}] \equiv t$ and $t[\sigma][\rho] \equiv t[\sigma \circ \rho]$, with analogous equations for types. These give associative substitution composition and identity laws, up to judgmental equality.

**Proof.** Induct on syntax, renaming binders fresh as necessary. Variables reduce the statements to lookup and the definition of componentwise composition. Application follows from the induction hypotheses on its parts. Under a binder, extend the substitution by mapping the freshly bound variable to itself; the single-variable composition lemma proves compatibility with that extension. Dependent constructors require the same argument simultaneously for their term and type components. Composition of substi-

tutions is associative because each of its components satisfies this substitution equation; identity laws follow from variable lookup. □

**Example 32 · A dependent tuple**
Suppose $\Gamma = (n : \mathrm{Nat}, v : \mathrm{Vec}(A, n))$. A substitution from an empty context can supply a numeral $k$ and a closed vector $w : \mathrm{Vec}(A, k)$. An arbitrary vector with a different length is not a valid second component. Applying another substitution to a context containing $A$ must substitute into $A$, $k$, and $w$ consistently.

An implementation may store the suspended expression $t[\sigma]$ rather than traverse $t$ immediately. Its rules include variable lookup, distribution through application, and binder lifting. The equations above specify the intended behavior. Adding a reduction rule that reassociates substitutions in both directions could obviously create loops; a concrete implementation must choose orientations and prove its own metatheory.

**Example 33 · Composition with a dependent tuple**
Suppose a substitution into $n : \mathrm{Nat}, v : \mathrm{Vec}(A, n)$ supplies terms $k, w$ over a context $\Delta$. Applying $\rho : \Theta \to \Delta$ produces $k[\rho]$ and $w[\rho]$. The latter has type $\mathrm{Vec}(A[\rho], k[\rho])$, so both the index and the element type must be reindexed. Keeping the old $k$ while substituting only into $w$ would generally be ill typed.

This bookkeeping explains the direction of context arrows: a substitution into a context is a way to provide its variables. Reindexing its types and terms therefore travels back along that arrow. Draw the arrow directions before expanding a nested substitution.

**Checkpoint 7.1.** If $\sigma : \Delta \to \Gamma$ and $\rho : \Theta \to \Delta$, in which context does $t[\sigma][\rho]$ live when $t$ originally lives in $\Gamma$?

**Takeaways.** Typed substitutions are the arrows between contexts. They make dependent-variable bookkeeping explicit and lead directly to semantic reindexing.
Continue with study guide U7 and exercise sheet U7.

## 7.2 Lecture 14 · Meaning explanations and a family-of-sets model

**Learning goals and prerequisites**
Explain canonical evidence, interpret contexts as environments, and prove soundness of the main universe-free rules.

**Recall and approach.** Recall functions, indexed families of sets, and sections selecting one element in each appropriate fiber.

A meaning explanation describes what counts as an element of a type and when two such elements agree. A natural number is given canonically by zero or successor data; a pair by its two components; a function by its action on arbitrary inputs. Elimination rules explain how this evidence may be used, and computation rules explain what happens when an eliminator meets a constructor. This is a semantic motivation, not a substitute for the formal metatheorems of a particular calculus.

**Definition 21 · Family-of-sets interpretation**
Interpret a context $\Gamma$ as a set of environments. A type over it is a family of sets $A(\gamma)$. A term is a section assigning $t(\gamma) \in A(\gamma)$. Context extension is the set of pairs $(\gamma, a)$ with $a \in A(\gamma)$. A substitution is a function between environment sets, and reindexing is composition: $A[\sigma](\delta) = A(\sigma(\delta))$.

Interpret a dependent product at $\gamma$ as the set of functions assigning to each $a \in A(\gamma)$ an element of $B(\gamma, a)$. Interpret a dependent sum as the corresponding set of pairs. Interpret natural numbers by the ordinary natural-number set with recursion and induction. Interpret an identity type as a singleton when its endpoints are equal and as the empty set otherwise.

**Proof strategy.** Interpret each rule in environments and prove its computation equation by evaluating the corresponding semantic construction.

**Theorem 23 · Soundness of the universe-free rules**
Valid typing derivations in the described fragment denote elements of their interpreted types; judgmental equalities denote equal elements or equal families.

**Proof.** Proceed by simultaneous induction on derivations. A variable is an environment projection. Abstraction maps each environment to the function given by the body section, and application evaluates it, so the beta equation is literal function evaluation. Pairing forms a member of the dependent sum, and projections recover its components, proving their computation equations. Natural-number elimination is interpreted by ordinary recursion with the supplied motive and step, which satisfies the two recursion equations. For identity elimination, an inhabited identity fiber has equal endpoints and its unique proof, so define the interpretation by the supplied reflexive case; the $J$ computation follows. Substitution is composition with environment maps, so its laws follow from function composition. Conversion is valid because equal interpreted types have the same elements. □

This is a model of the universe-free fragment. Interpreting the entire universe hierarchy additionally requires externally chosen collections of small sets closed under the relevant operations; we do not pretend that the set of all sets is itself a set. The model also validates principles, such as uniqueness of identity proofs, that our intensional syntax does not adopt. Soundness only says derivable statements are valid in the model, not that every model-valid statement is derivable.

**Example 34 · An empty fiber distinguishes sums from products**
Let the base set have two elements and let one fiber be empty while the other is a singleton. The dependent sum has one element, from the inhabited fiber. The dependent product is empty, because a section would have to select an element of the empty fiber as well.

This interpretation mirrors existential and universal evidence: one successful witness is enough for a sum, whereas a product must work at every input. It also illustrates a reliable semantic counterexample method. To refute a proposed uniform constructive map, look for an interpretation in which its domain is inhabited and its codomain empty, then invoke soundness in the forward direction.

**Checkpoint 7.2.** What is a term of a family $A(\gamma)$ in this model?

**Takeaways.** Separate syntax, judgmental equality, internal identity types, and semantic equality. The set model helps calculate meanings while illustrating that a model may identify more than the syntax does.
Continue with study guide U7 and exercise sheet U7.

# 8 Unit 8 · Categorical models and proof assistants

## 8.1 Lecture 15 · Products, exponentials, and comprehension

**Learning goals and prerequisites**
Interpret simple types in a Cartesian closed category and describe the extra structure needed for dependent types.

**Recall and approach.** Recall products, functions between sets, and reindexing of a family along a context map.

The categorical concepts needed here are introduced within this lecture. A terminal object admits one map from every object. A product $A \times B$ represents pairs of maps into $A, B$. An exponential $B^A$ represents maps out of a product through a natural bijection $\mathcal{C}(X \times A, B) \equiv \mathcal{C}\big(X, B^A\big)$. A category with finite products and such exponentials is Cartesian closed.

**Definition 22 · Simple-type interpretation**
Interpret base types as objects, function types as exponentials, and a context as a finite product. A judgment $\Gamma \vdash t : A$ denotes a morphism from the context object to the type object. Variables are projections, abstraction is exponential transposition, and application uses evaluation after pairing the function and argument maps.

**Proof strategy.** Use evaluation and currying for simple beta soundness; prove comprehension by checking and uniquely determining both coordinates.

**Theorem 24 · Beta soundness in a Cartesian closed category**
The interpretation of a beta contraction is an equality of morphisms.

**Proof.** If the body denotes $b : \Gamma \times A \to B$, abstraction denotes its transpose $\Lambda(b) : \Gamma \to B^A$. Applying this function to an argument $u : \Gamma \to A$ denotes $\mathrm{ev} \circ \langle \Lambda(b), u \rangle$. The defining transpose/evaluation equation makes this equal to $b \circ \langle 1_\Gamma, u \rangle$, which is exactly the interpretation of substituting $u$ into the body.□

The exponential universal property also validates function eta semantically. Our minimal paper syntax need not make eta a judgmental rule; the model may satisfy additional equalities. This is another reason not to read semantic equality backward as a derivation principle.

Dependent types require a category of contexts together with types and terms over each context, reindexing along substitutions, and context extension. A convenient formulation is a category with families. Its comprehension property can be described without additional machinery.

**Definition 23 · Context comprehension**
For a type $A$ over $\Gamma$, the extended context $\Gamma \cdot A$ has a projection $p : \Gamma \cdot A \to \Gamma$ and a generic term $q : A[p]$. For every substitution $\sigma : \Delta \to \Gamma$ and term $t : A[\sigma]$ in $\Delta$, there is a unique pairing $\langle \sigma, t \rangle : \Delta \to \Gamma \cdot A$ with $p \circ \langle \sigma, t \rangle = \sigma$ and $q[\langle \sigma, t \rangle] = t$.

**Theorem 25 · Comprehension in families of sets**
The environment interpretation from Lecture 14 has this property.

**Proof.** The extended context is the set of pairs $(\gamma, a)$. Projection forgets the second coordinate, and the generic term returns it. Given $\sigma, t$, define the pairing by $\delta \mapsto (\sigma(\delta), t(\delta))$. The type of $t$ makes this a

valid pair, and the two component equations hold. Any map with those equations has exactly these two coordinates and is therefore equal, proving uniqueness. □

Dependent products, sums, and identity types add further operations to this contextual structure, stable under reindexing. In the family model they are built fiber by fiber as in Lecture 14. More sophisticated models can retain richer identity information, but those developments are outside the chosen proof-assistant application.

**Example 35 · The empty context as a categorical unit**
In the category of syntactic contexts and substitutions, a map into the empty context supplies an empty tuple and is unique. Thus the empty context is terminal. A closed term of a closed type is a map from this terminal context into its one-variable extension. In the set model, the empty context is interpreted by a singleton environment, and such a map selects one element of the interpreted type.

The direction is worth checking: context morphisms provide the target variables, so the empty target is easiest to supply. This connects the "global elements are maps from one" idea from category theory with the distinction between closed terms and terms carrying assumptions.

**Checkpoint 8.1.** Why is an ordinary Cartesian closed category alone insufficient to specify all dependent types?

**Takeaways.** Simple functions are modeled by exponentials; dependency is modeled by families and comprehension. Category Theory provides optional further practice with these universal properties.
Continue with study guide U8 and exercise sheet U8.

## 8.2 Lecture 16 · Formal proofs as checked programs

**Learning goals and prerequisites**
Translate paper derivations into Agda, interpret checker feedback mathematically, and distinguish verified code from unproved metatheory.

**Recall and approach.** Recall introduction/elimination rules, structurally recursive induction, and transport between dependent fibers.

The four optional labs use a small common mathematical fragment. They define their own basic datatypes and need no external standard library. Completed solutions are checked with the project's pinned Agda release using safe mode and without the K principle. Starter files contain deliberately unfinished goals; only the completed solutions are expected to pass the verification script.

**Example 36 · From a theorem to a function**
To prove $A \to B \to A$, write a function taking inputs of types $A$, $B$ and returning the first. To prove a natural-number identity, pattern-match on the number and use the recursively obtained equality. To preserve vector length, give a result type indexed by the same length. To change a dependent fiber along equality, use transport. Each technique corresponds to one of the preceding introduction or elimination rules.

Lab 1 treats implication, products, sums, and absurd elimination. Lab 2 defines addition recursively and proves right-zero and associativity by induction. Lab 3 defines length-indexed vectors, mapping, and append with its result index. Lab 4 defines equality elimination operations and transport, showing the distinction between a computation that closes by reflexivity and a proof that must inspect equality evidence.

**Definition 24 · Verification contract**
For a completed lab accepted under the recorded settings, its declarations have the stated types according to that Agda checker, including its termination, positivity, and universe checks.

This is the contract of the checking procedure, not an independent proof of the compiler's correctness. The project invokes the checker on the completed source files and records its successful exit status. Safe mode excludes the mechanisms used to assert an arbitrary theorem without a proof, and the files contain no unresolved goals. Thus the result is evidence for these concrete formal declarations under the stated trusted toolchain.

**Theorem 26 · A vector program preserves identity**
For every vector $v : \mathrm{Vec}(A, n)$, there is a proof of $\mathrm{Id}_{\mathrm{Vec}(A,n)}(\mathrm{map}(\lambda x.x, v), v)$.

**Proof strategy.** Induct on the vector and lift the tail equality through the constructor with its head fixed.
**Proof.** For nil, map computes to nil, so reflexivity supplies the required proof. For $v = \mathrm{cons}(a, w)$, map computes to $\mathrm{cons}(a, \mathrm{map}(\lambda x.x, w))$. The induction hypothesis identifies the mapped tail with $w$ in its fixed length fiber. Congruence under $z \mapsto \mathrm{cons}(a, z)$ gives equality with $\mathrm{cons}(a, w)$ in the successor-length fiber. These are all vector constructors. The completed Lab 3 includes this proof as `map-identity`, connecting each clause to this induction. □

When a term fails to check, inspect the expected type and the inferred type before changing the program. A vector-length mismatch often means a recursively defined addition expression has not reduced in the direction you expected. An equality goal that fails under reflexivity may require induction or transport. An unsolved implicit argument may mean the intended type is ambiguous rather than false.

**Watch for this.** Type checking a collection of examples does not prove normalization or consistency of the entire proof assistant. Our explicit normalization proof concerned the arrow-only simply typed core. The richer dependent theory and the checker require their own metatheoretic justification.

For independent study, first solve each lab on paper, then fill the starter functions, then run the checker. Compare with the completed solution only after understanding the remaining goal. Keep the `.agda` files as executable companions; the lectures and exercises remain fully usable without installing an editor plugin.

**Example 37 · A checked theorem can still be the wrong theorem**
If the intended goal is addition commutativity but the written type is reflexivity of $m + n$ with itself, a reflexivity proof checks without establishing commutativity. The remedy is to compare the encoded signature with the intended statement before reading the implementation. For vector append, inspect the length index; for a transport theorem, inspect both endpoint fibers; for a logical implication, inspect the order of its assumptions.

The laboratory workflow therefore has three checks: verify the statement on paper, complete its proof term under the fixed rules, and explain the accepted clauses mathematically. Machine checking is strongest when paired with that semantic audit of what was actually asked.

**Checkpoint 8.2.** Why should a proof with an unfilled hole not count as verified merely because the surrounding file parses?

**Takeaways.** The proof assistant makes the typing rules operational. Its most useful feedback is a precise statement of the mathematical obligation still missing.
Continue with study guide U8 and exercise sheet U8.